SMARTBRIDGE
Let's Bridge the Gap

SKILL WALLET
A SmartBridge Product

Project Executable Files

Date	1 July 2026
Team ID	
Project Name	Rising Waters
Maximum Marks	3 Marks

Project Executable Files
This section includes all the executable and supporting files required for the Flood Prediction System. The project files are organized to enable the evaluator to install, run, and verify the Machine Learning model and Flask web application using the provided source code, dependencies, and execution instructions.

Step 1: Submission Checklist

S.No	**Item to Submit**	**Submitted (Yes / No / NA)**
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Database schema / seed files (if applicable)	N/A
5	Environment configuration file (.env.example or similar)	No
6	Deployed application URL (if hosted)	Yes
7	APK / Executable binary (if applicable for mobile/desktop apps)	N/A
8	Dockerfile / Containerization config (if applicable)	N/A
9	Test files and test results	Yes
10	Demo video or walkthrough (if required)	Yes

SMARTBRIDGE
Let's Bridge the Gap

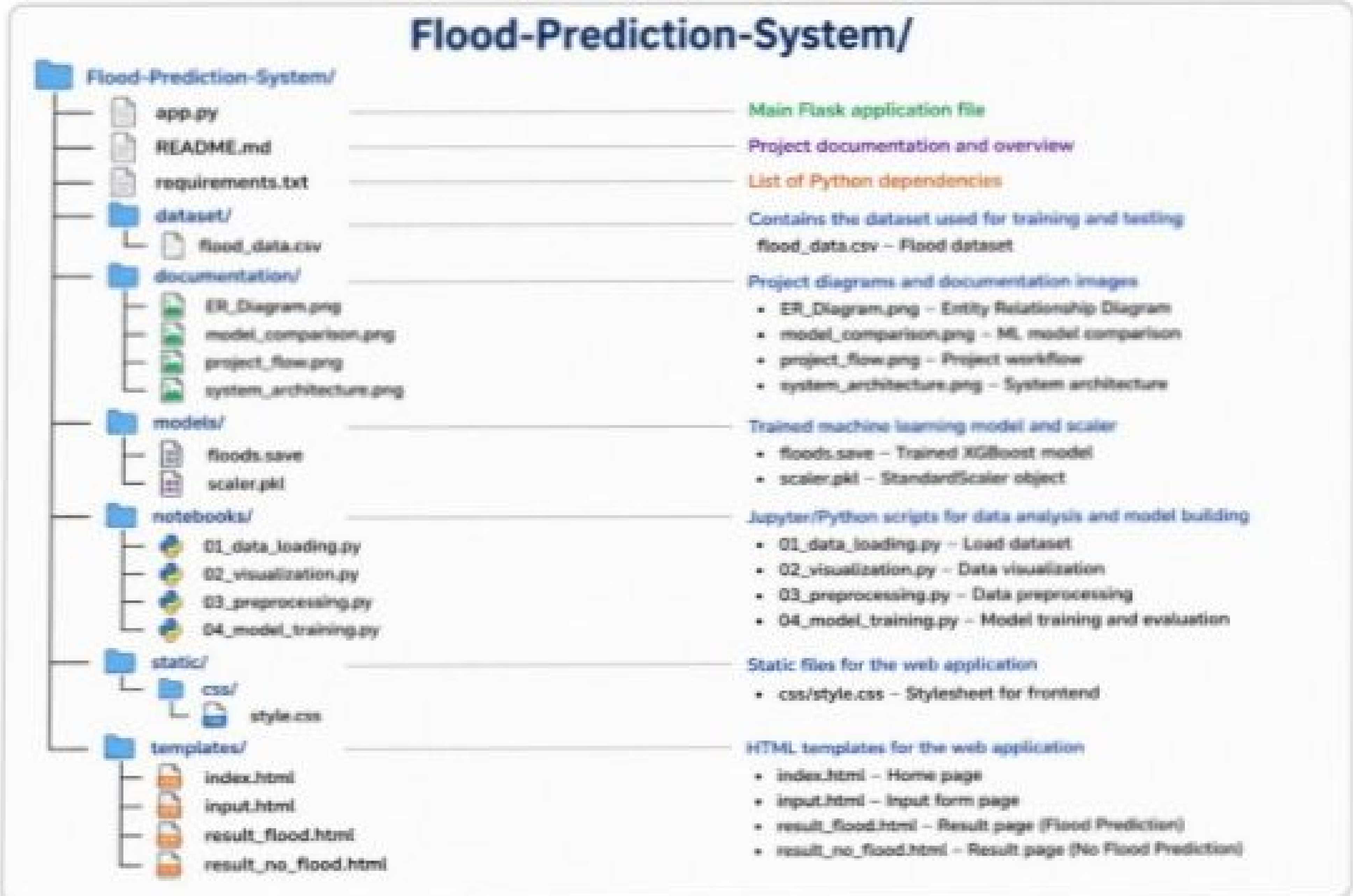
SKILL WALLET
A SmartBridge Product

Step 2: File / Folder Structure

The following section presents the file and folder structure of the Flood Prediction System, highlighting the organization of the source code, machine learning models, datasets, documentation, and supporting project files.

Flood-Prediction-System/

```
|— app.py
|— README.md
|— requirements.txt
|
├── dataset/
|    └── flood_data.csv
|
├── documentation/
|    ├── ER_Diagram.png
|    ├── model_comparison.png
|    ├── project_flow.png
|    └── system_architecture.png
|
├── models/
|    ├── floods.save
|    └── scaler.pkl
|
├── notebooks/
|    ├── 01_data_loading.py
|    ├── 02_visualization.py
|    ├── 03_preprocessing.py
|    └── 04_model_training.py
|
├── static/
|    └── css/
|         └── style.css
|
└── templates/
     ├── index.html
     ├── input.html
     ├── result_flood.html
     └── result_no_flood.html
```


Flood-Prediction-System/
Flood-Prediction-System/
app.py
Main Flask application file
README.md
Project documentation and overview
requirements.txt
List of Python dependencies
dataset/
Contains the dataset used for training and testing
flood_data.csv
flood_data.csv – Flood dataset
documentation/
Project diagrams and documentation images
ER_Diagram.png
model_comparison.png
project_flow.png
system_architecture.png
ER_Diagram.png – Entity Relationship Diagram
model_comparison.png – ML model comparison
project_flow.png – Project workflow
system_architecture.png – System architecture
models/
Trained machine learning model and scaler
floods.save
scaler.pkl
floods.save – Trained XGBoost model
scaler.pkl – StandardScaler object
notebooks/
Jupyter/Python scripts for data analysis and model building
01_data_loading.py
02_visualization.py
03_preprocessing.py
04_model_training.py
01_data_loading.py – Load dataset
02_visualization.py – Data visualization
03_preprocessing.py – Data preprocessing
04_model_training.py – Model training and evaluation
static/
css/
style.css
Static files for the web application
css/style.css – Stylesheet for frontend
templates/
HTML templates for the web application
index.html
input.html
result_flood.html
result_no_flood.html
index.html – Home page
input.html – Input form page
result_flood.html – Result page (Flood Prediction)
result_no_flood.html – Result page (No Flood Prediction)

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	https://rising-waters-hh79.onrender.com/
Login Credentials (Demo)	Not Required
Platform / Hosting Provider	Render
Repository Link	https://github.com/Teerdhana07/Rising-waters
Demo Video Link	https://drive.google.com/file/d/19lvLWJkaWydopGWNJsB8Y-s9QYOrdfFQ/view?usp=sharing

Step 4: Run Instructions

The following instructions explain how to run the Flood Prediction System locally and generate flood predictions using the Flask web application and trained XGBoost model.

Pre-requisites

- Python 3.10+
- pip
- Internet Browser

Steps

1. Clone/download the **Rising Waters** project.
2. Open the project folder.
3. Install dependencies: **pip install -r requirements.txt**
4. Run the application: **python app.py**
5. Open browser at**: http://127.0.0.1:5000**
6. Enter the required weather parameters and click **Predict** to view the flood prediction result.

Step 4: Run Instructions

The following instructions explain how to run the Flood Prediction System locally and
generate flood predictions using the Flask web application and trained XGBoost model.

Pre-requisites:

- Python 3.10+
- pip
- Internet Browser

Steps:

Step 1: Clone/download the Rising Waters project.

```
git clone https://github.com/KSriMounika/Rising-Waters
```

Step 2: Open the project folder.

Step 3: Install dependencies:

```
pip install -r requirements.txt
```

Step 4: Run the application:

```
python app.py
```

Step 5: Open browser at: http://127.0.0.1:5000

Step 6: Enter the required weather parameters and click Predict to view the flood prediction result.

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	Application is not deployed online	Can be deployed using Render, Vercel, AwS or Azure
2	Predictions depend on the quality of the historical dataset	Use larger and updated datasets for improved accuracy
3	No real-time weather API integration	Can be added in future versions